Step – 1 Monolith vs Microservices Architecture

Monolithic architecture: is a traditional software architecture where an entire application is built and deployed as **one single unit**.

Microservice Architecture is a software design approach where an application is divided into **small, independent services**.

Modular monolith is a software architecture where an application is deployed as **one unit**, but internally it is divided into **independent, well-organized modules**.

Distributed systems are systems where multiple computers work together and communicate over a network to achieve a common goal.

Service boundaries define **what each microservice is responsible for**. Each service should focus on one specific business capability and communicate with other services through well-defined APIs.

Independent deployment means each microservice can be **developed, tested, and deployed separately** without affecting other services.

Scalability means the ability of a system or application to **handle increasing users, data, or workload without affecting performance**.

Step- 2 Creating architecture Diagrams

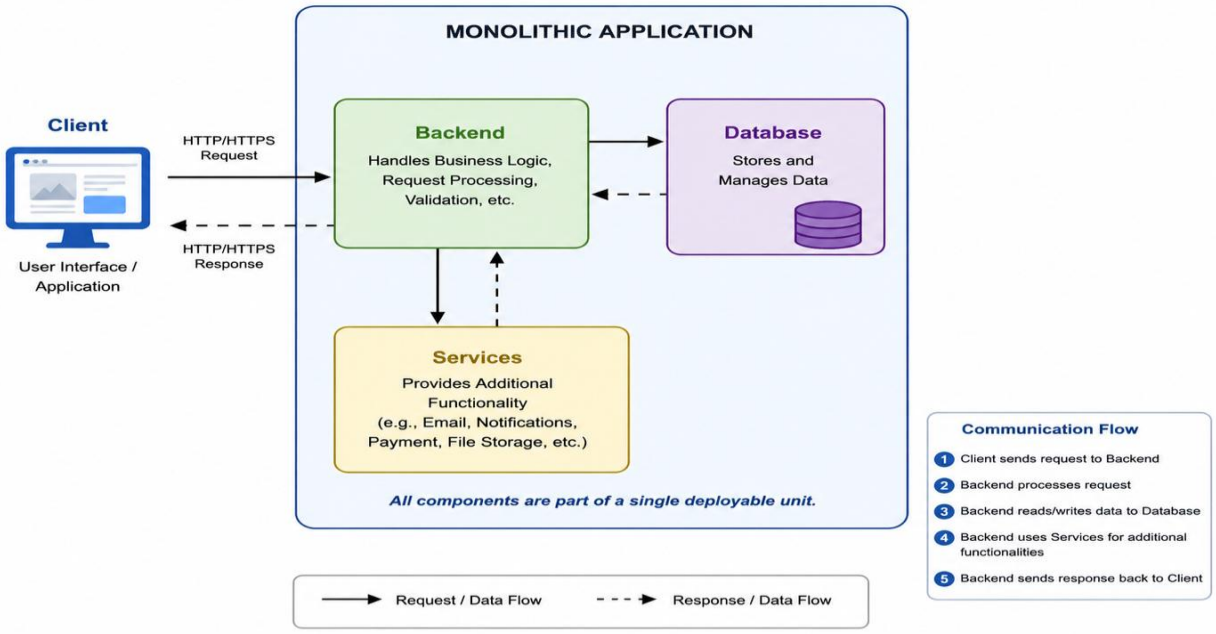
Monolithic application:

Client → Backend → Database

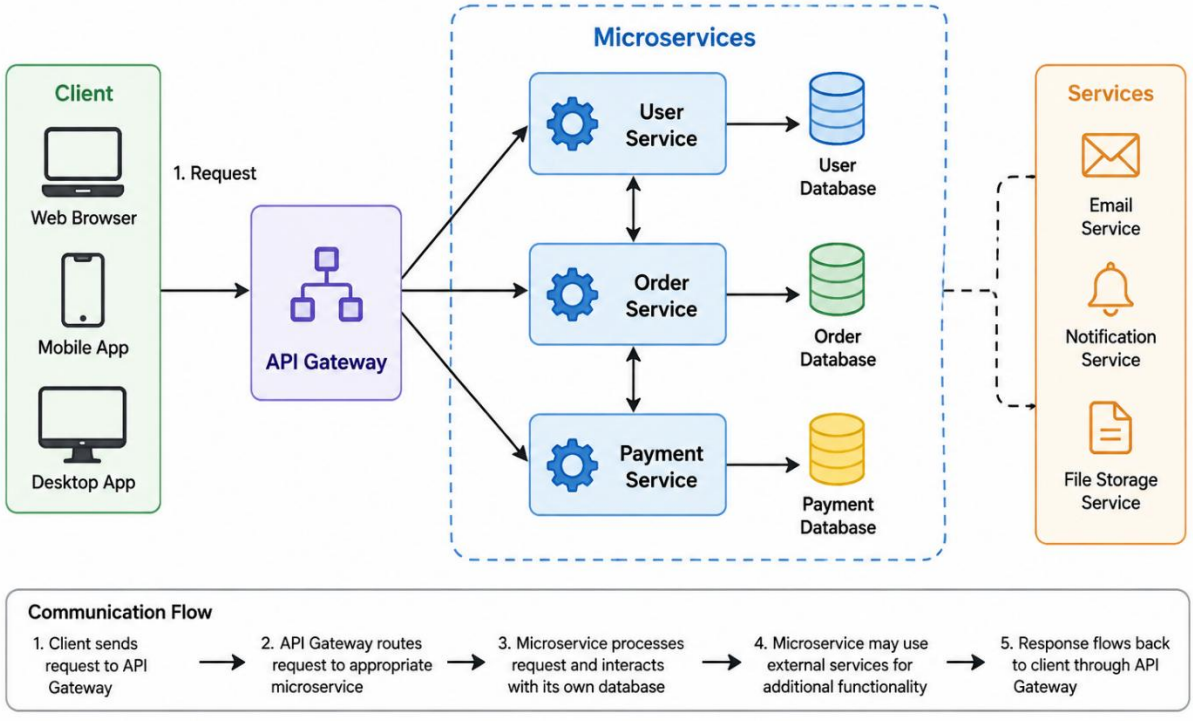
↕

Services

Monolithic Application Architecture

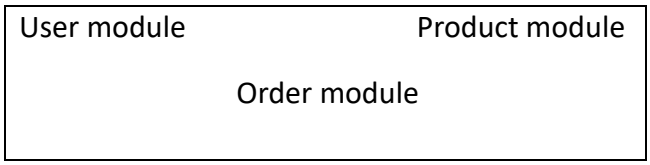


Microservices Architecture Diagram



Step – 3 **Spring Boot monolithic application**.

Simple architecture: All modules should be inside **one single Spring Boot application**



User, Product, and Order modules are developed and deployed together as **one monolithic application**.

simple **Spring Boot monolithic Java application** with **User, Product, and Order modules**.

Project structure

src/main/java/com/example/monolith/

── MonolithApplication.java

── user/

   ── UserController.java

── product/

  ── ProductController.java

── order/

  └── OrderController.java

Main Spring Boot application

```
package com.example.monolith;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

public class MonolithApplication {

    public static void main(String[] args) {

      SpringApplication.run(MonolithApplication.class, args);

  }

}
```

User Module

```
package com.example.monolith.user;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
public class UserController {
   public String getUsers() {
      return "User Module Working";
   }
}
```

Product Module

```
package com.example.monolith.product;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
public class ProductController {
   public String getProducts() {
      return "Product Module Working";
   }
}
```

Order Module

```
 package com.example.monolith.order;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
public class OrderController {
   public String getOrders() {
      return "Order Module Working";
   }        }
```

Step – 4 Identify problems

Document problems: As the application becomes large, it becomes difficult to maintain clear and updated documentation.

Deployment problems: Even a small change requires deploying the entire application, which takes more time and can cause issues.

Scaling problems: You must scale the whole application, even if only one feature needs more resources.

Database coupling: All modules use the same database, so changes in one module can affect other modules.

Code coupling: Different parts of the application are closely connected, making changes difficult.

Team dependency: Multiple teams working on the same application may depend on each other, causing delays.

Failure impact: If one important part of the application fails, it can affect or stop the entire application.

Step 5 – Convert the Design

Redesign the same application: Change the existing application design into a better and more flexible architecture.

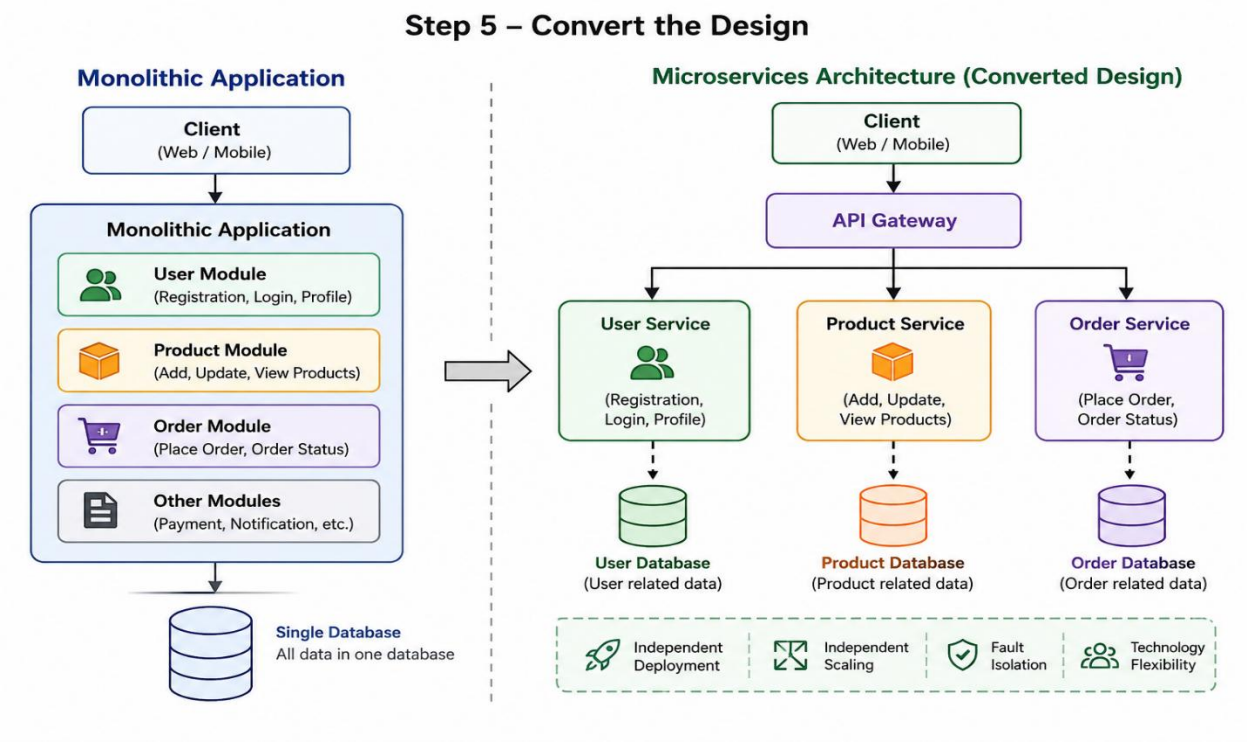
User Service: Handles user-related features such as registration, login, and user details.

Product Service: Manages product-related features such as adding, updating, viewing, and deleting products.

Order Service: Handles customer orders, order processing, and order status.

Conversion:

- **User-related functionality → User Service**
- **Product-related functionality → Product Service**
- **Order-related functionality → Order Service**
- Each service has its **own database** and can be **developed, deployed, and scaled independently**.



Step 6 – Compare Both Architectures

Create a document comparing **Monolithic Architecture** and **Microservices Architecture**.

Topic	Monolithic Architecture	Microservices Architecture
Development	All features are developed in one application.	Each service can be developed independently.
Deployment	The entire application must be deployed together.	Each service can be deployed separately.
Scaling	The whole application must be scaled.	Only the required service can be scaled.
Maintenance	Difficult as the application becomes large.	Easier because services are smaller and separate.
Testing	Testing the whole application can take more time.	Each service can be tested independently.
Failure Handling	One failure can affect the entire application.	Failure in one service has less impact on other services.

Step 7 – Testing

Explanation how each architecture behaves when testing :

- **User service fails:**
    - **Monolithic:** The entire application may be affected.
    - **Microservices:** Only the User Service may be affected; other services can continue.
- **Database fails:**
    - **Monolithic:** The whole application may stop because it depends on one database.
    - **Microservices:** Only the service connected to the failed database may be affected.
- **Order service receives heavy traffic:**
    - **Monolithic:** The whole application may need to be scaled.
    - **Microservices:** Only the Order Service can be scaled to handle more traffic.